Name : M SHUMAIL

Roll no : CYS-18

Section C

Subject :

DSA (data structure and algorithm)

Task A

As a class, implement the complete Library Returned-Book Processing System described above.

Students must implement and demonstrate all major Stack operations:

Push()

Pop()

GetTopValue()

IsEmpty()

IsNotEmpty()

PrintStack()

Constructor

Copy Constructor

Assignment Operator

Destructor

The implementation must use dynamically allocated nodes and pointers.

Code :

```
#include <iostream>
#include <string>

using namespace std;

// 1. BookNode Structure
struct BookNode {
  int bookID;
  string title;
  string author;
  BookNode* next;

  // Constructor to set values easily
  BookNode(int id, string t, string a) {
    bookID = id;
    title = t;
    author = a;
    next = nullptr;
  }
};

// 2. Stack Class
class Stack {
private:
  BookNode* top;
  int count;

public:
```

```
// Constructor
Stack() {
  top = nullptr;
  count = 0;
}

// Destructor (Frees memory automatically)
~Stack() {
  while (!IsEmpty()) {
    BookNode* temp = Pop();
    delete temp;
  }
}

// Copy Constructor (Deep Copy)
Stack(const Stack& other) {
  top = nullptr;
  count = 0;
  BookNode* curr = other.top;
  BookNode* tail = nullptr;

  while (curr != nullptr) {
    BookNode* newNode = new BookNode(curr->bookID, curr->title, curr->author);
    if (top == nullptr) {
      top = newNode;
    } else {
      tail->next = newNode;
    }
    tail = newNode;
```

```
        curr = curr->next;
    }
    count = other.count;
}

// Assignment Operator Overload
Stack& operator=(const Stack& other) {
    if (this != &other) {
        // Delete old memory
        while (!IsEmpty()) {
            delete Pop();
        }
        // Copy new elements
        BookNode* curr = other.top;
        BookNode* tail = nullptr;

        while (curr != nullptr) {
            BookNode* newNode = new BookNode(curr->bookID, curr->title, curr->author);
            if (top == nullptr) {
                top = newNode;
            } else {
                tail->next = newNode;
            }
            tail = newNode;
            curr = curr->next;
        }
        count = other.count;
    }
    return *this;
```

```
}

// 1. Push - Place book on top
void Push(BookNode* book) {
   if (book == nullptr) return;
   book->next = top;
   top = book;
   count++;
   cout << "[PUSH] Added: " << book->title << endl;
}

// 2. Pop - Remove top book
BookNode* Pop() {
   if (IsEmpty()) {
      cout << "[ERROR] Pile is empty!\n";
      return nullptr;
   }
   BookNode* temp = top;
   top = top->next;
   temp->next = nullptr;
   count--;
   return temp;
}

// 3. GetTopValue - View top book
BookNode* GetTopValue() {
   return top;
}
```

```
    // 4. IsEmpty
    bool IsEmpty() {
        return top == nullptr;
    }

    // 5. IsNotEmpty
    bool IsNotEmpty() {
        return top != nullptr;
    }

    // 6. PrintStack - Display all books
    void PrintStack() {
        if (IsEmpty()) {
            cout << "Pile is empty.\n";
            return;
        }

        cout << "\n--- RETURNED BOOKS PILE (Total: " << count << ") ---\n";
        BookNode* temp = top;
        while (temp != nullptr) {
            cout << "ID: " << temp->bookID
                 << " | Title: " << temp->title
                 << " | Author: " << temp->author << "\n";
            temp = temp->next;
        }
        cout << "----------------------------------------\n";
    }
};
```

```
// Main Function
int main() {
    Stack pile;

    // Adding books
    pile.Push(new BookNode(101, "Programming Fundamentals", "Author A"));
    pile.Push(new BookNode(102, "Object Oriented Programming", "Author B"));
    pile.Push(new BookNode(103, "Database Systems", "Author C"));
    pile.Push(new BookNode(104, "Data Structures & Algorithms", "Author D"));

    // Display stack
    pile.PrintStack();

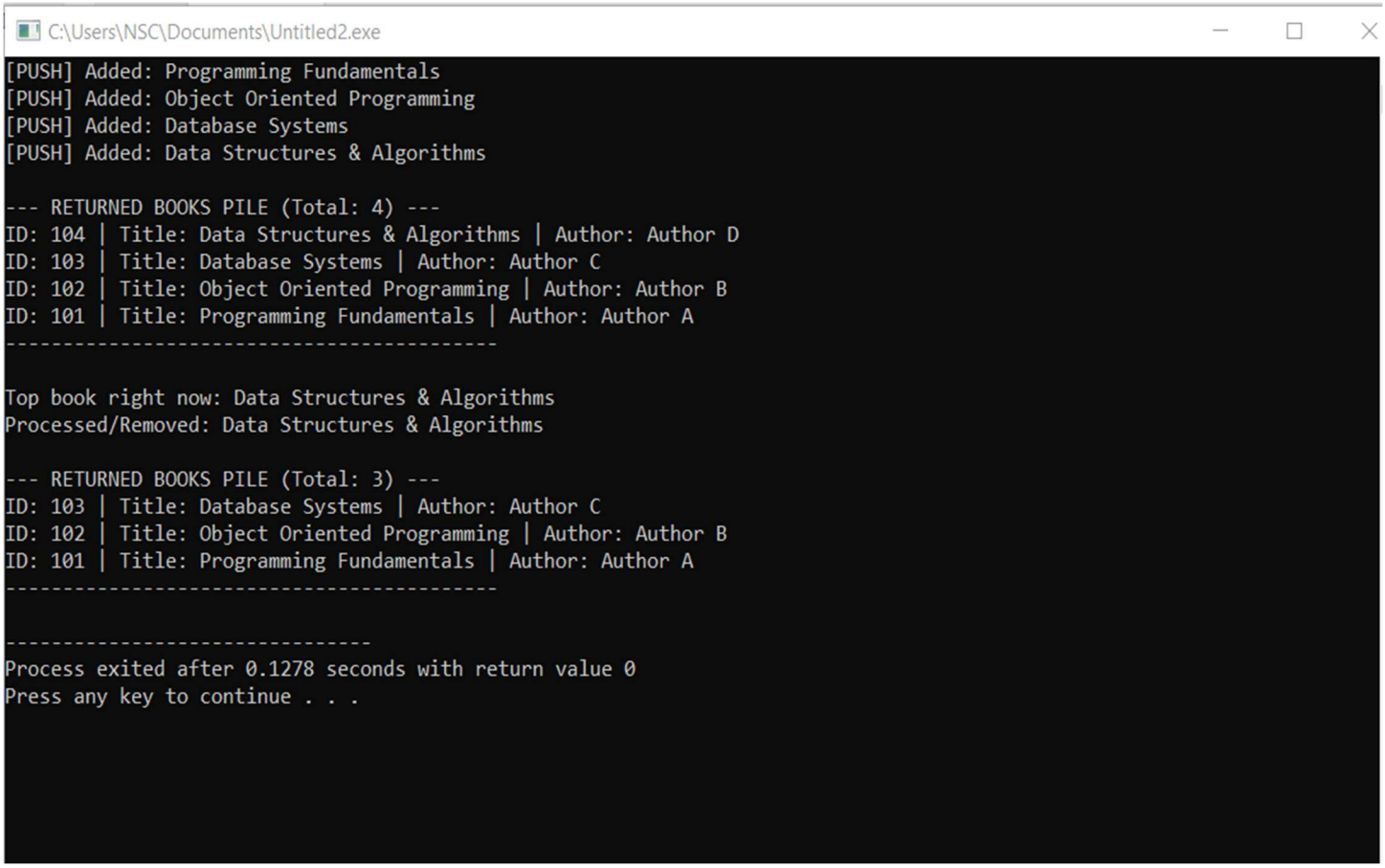
    // View top book
    BookNode* topBook = pile.GetTopValue();
    if (topBook) {
        cout << "\nTop book right now: " << topBook->title << endl;
    }

    // Remove top book
    BookNode* removed = pile.Pop();
    if (removed) {
        cout << "Processed/Removed: " << removed->title << endl;
        delete removed; // Memory clear
    }

    // Display remaining stack
    pile.PrintStack();
```

```
    return 0;
}
```

Output:



Part B:

Apply the Stack data structure to the student's own project.

Students must identify a situation within their individual project where:

The most recently added item needs to be accessed or removed first.

They should then:

1. Identify the real-world problem.

2. Explain why LIFO is required.

3. Design a suitable node structure.

4. Implement a Stack class.

5. Implement the required Stack operations.

6. Demonstrate the Stack using their project's own data.

For the individual project students must identify a genuine Stack-based requirement within

their own project.

Answer:

Real-World Problem Statement

In urban facilities, driveways are frequently constrained to narrow, single-lane parking alleys where vehicles park sequentially one behind another. Entry and exit occur through the exact same lane access point. When a driver wants to retrieve a parked vehicle, vehicles parked further down the lane cannot bypass vehicles parked in front towards the exit gate.

Why LIFO (Stack) is Required?

This physical constraint strictly enforces **Last-In, First-Out (LIFO)** operational logic:

- **Last-In:** The vehicle arriving most recently is parked closest to the entrance/exit access point.

- **First-Out:** To leave the parking lane, the vehicle closest to the exit (the last one parked) must be removed first before any previously parked vehicle can leave.

Because direct access to vehicles in the middle is physically impossible without moving outer vehicles first, the **Stack Data Structure** is the most accurate computer representation for this system.

Data Structure Architecture

- **`VehicleNode` Structure:** Represents a single vehicle containing data attributes (`plateNumber`, `ownerName`, `vehicleType`) and a pointer (`next`) pointing to the vehicle parked directly behind it.
- **`ParkingStack` Class:** Manages the `top` pointer (vehicle closest to the exit) and implements required operations (`Push`, `Pop`, `GetTopValue`, `IsEmpty`, `IsNotEmpty`, `PrintStack`) along with dynamic memory management (`Destructor`, `Copy Constructor`, and `Assignment Operator`).

Code :

```
#include <iostream>
#include <string>


using namespace std;


// 1. VehicleNode Structure
struct VehicleNode {
    string plateNumber;
    string ownerName;
```

```
    string vehicleType;
    VehicleNode* next;

    // Easy Constructor
    VehicleNode(string plate, string owner, string type) {
        plateNumber = plate;
        ownerName = owner;
        vehicleType = type;
        next = nullptr;
    }
};

// 2. ParkingStack Class
class ParkingStack {
private:
    VehicleNode* top;
    int count;

public:
    // Constructor
    ParkingStack() {
        top = nullptr;
        count = 0;
```

```
    }

    // Destructor (Cleans memory)
    ~ParkingStack() {
        while (!IsEmpty()) {
            VehicleNode* temp = Pop();
            delete temp;
        }
    }

    // Copy Constructor (Deep Copy)
    ParkingStack(const ParkingStack& other) {
        top = nullptr;
        count = 0;
        VehicleNode* curr = other.top;
        VehicleNode* tail = nullptr;

        while (curr != nullptr) {
            VehicleNode* newNode = new VehicleNode(curr->plateNumber, curr->ownerName, curr->vehicleType);
            if (top == nullptr) {
                top = newNode;
            } else {
                tail->next = newNode;
```

```
        }
        tail = newNode;
        curr = curr->next;
      }
      count = other.count;
    }

    // Assignment Operator Overload
    ParkingStack& operator=(const ParkingStack& other) {
      if (this != &other) {
        // Delete old memory
        while (!IsEmpty()) {
          delete Pop();
        }
        // Copy new elements
        VehicleNode* curr = other.top;
        VehicleNode* tail = nullptr;

        while (curr != nullptr) {
          VehicleNode* newNode = new VehicleNode(curr->plateNumber, curr->ownerName, curr->vehicleType);
          if (top == nullptr) {
            top = newNode;
          } else {
```

```
            tail->next = newNode;
        }
        tail = newNode;
        curr = curr->next;
      }
      count = other.count;
   }
   return *this;
}

// 1. Push - Park vehicle on top
void Push(VehicleNode* vehicle) {
   if (vehicle == nullptr) return;
   vehicle->next = top;
   top = vehicle;
   count++;
   cout << "[PARKED] " << vehicle->plateNumber << endl;
}

// 2. Pop - Remove last parked vehicle
VehicleNode* Pop() {
   if (IsEmpty()) {
      cout << "Lane is empty!\n";
```

```
        return nullptr;
    }
    VehicleNode* temp = top;
    top = top->next;
    temp->next = nullptr;
    count--;
    return temp;
}

// 3. GetTopValue - View exit blocking vehicle
VehicleNode* GetTopValue() {
    return top;
}

// 4. IsEmpty
bool IsEmpty() {
    return top == nullptr;
}

// 5. IsNotEmpty
bool IsNotEmpty() {
    return top != nullptr;
}
```

```
    // 6. PrintStack - Display all parked vehicles
    void PrintStack() {
        if (IsEmpty()) {
            cout << "Parking lane is empty.\n";
            return;
        }

        cout << "\n--- PARKING LANE STATUS (Total: " << count << ") ---\n";
        VehicleNode* temp = top;
        while (temp != nullptr) {
            cout << "Plate: " << temp->plateNumber
                 << " | Owner: " << temp->ownerName
                 << " | Type: " << temp->vehicleType << "\n";
            temp = temp->next;
        }
        cout << "-----------------------------------------\n";
    }
};

// Main Function
int main() {
    ParkingStack lane;
```

```
// Parking vehicles
lane.Push(new VehicleNode("LEA-1234", "Ali", "Car"));
lane.Push(new VehicleNode("LHR-5678", "Ahmed", "SUV"));
lane.Push(new VehicleNode("ICT-9012", "Usman", "Car"));

// Display lane
lane.PrintStack();

// View exit blocking vehicle
VehicleNode* exitCar = lane.GetTopValue();
if (exitCar) {
   cout << "\nCar near Exit: " << exitCar->plateNumber << endl;
}

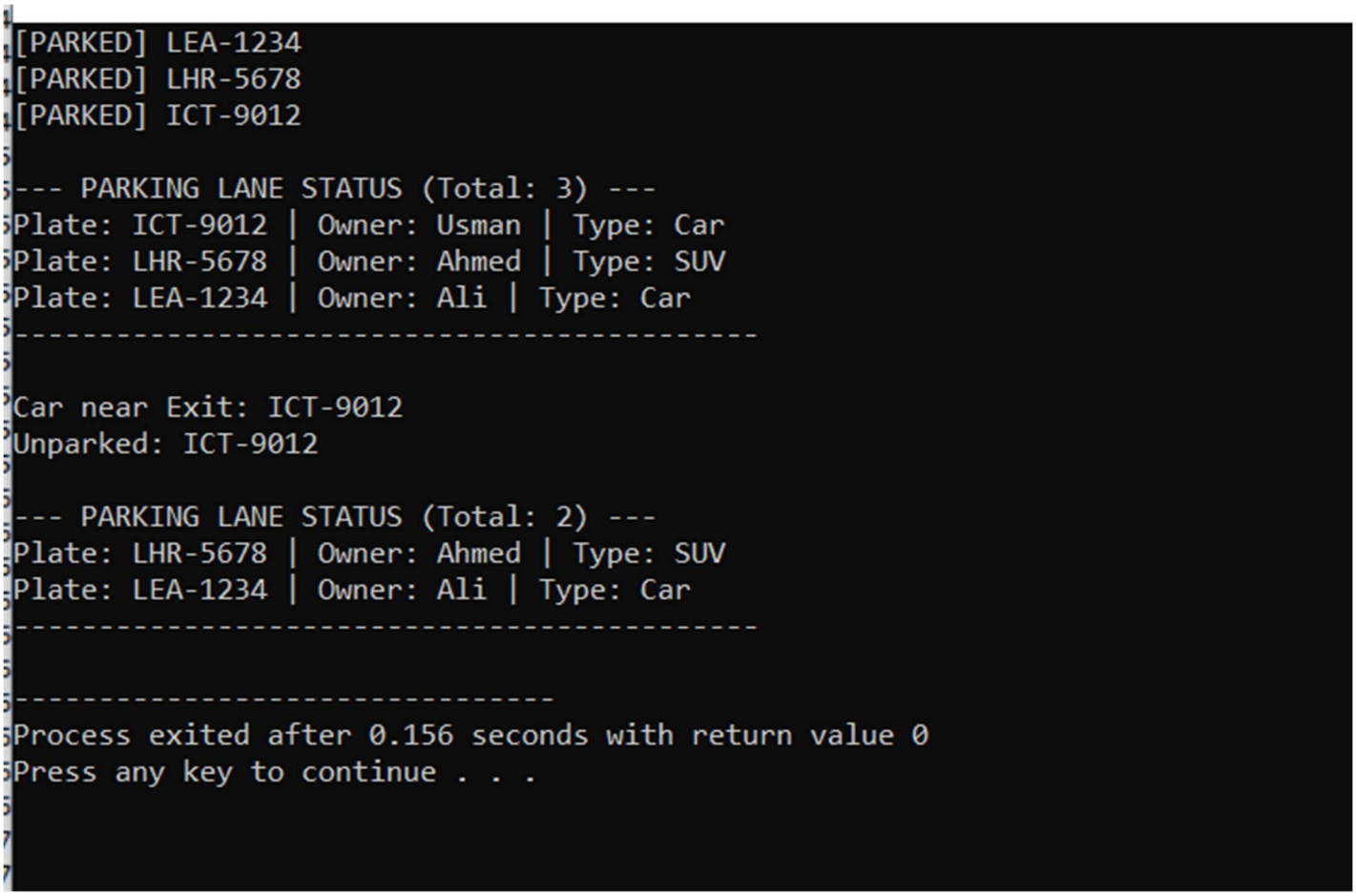
// Unpark vehicle
VehicleNode* unparked = lane.Pop();
if (unparked) {
   cout << "Unparked: " << unparked->plateNumber << endl;
   delete unparked; // Clean memory
}

// Display remaining lane
```

```
   Lane.PrintStack
();


   return 0;
}
```

Output:



```
[PARKED] LEA-1234
[PARKED] LHR-5678
[PARKED] ICT-9012

--- PARKING LANE STATUS (Total: 3) ---
Plate: ICT-9012 | Owner: Usman | Type: Car
Plate: LHR-5678 | Owner: Ahmed | Type: SUV
Plate: LEA-1234 | Owner: Ali | Type: Car
-----------------------------------------

Car near Exit: ICT-9012
Unparked: ICT-9012

--- PARKING LANE STATUS (Total: 2) ---
Plate: LHR-5678 | Owner: Ahmed | Type: SUV
Plate: LEA-1234 | Owner: Ali | Type: Car
-----------------------------------------


--------------------------------
Process exited after 0.156 seconds with return value 0
Press any key to continue . . .
```